

PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

SOFTWARE ENGINEERING - LEVEL 3 MINI-PROJECT
SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

for

Traffic Management System (Simulation)

Item	Detail
Project Title	Traffic Management System (Simulation)
Project ID	TBD
Group Number	Team 11
Course	UE24CS341A - Software Engineering
Semester	V Semester
Development Methodology	Agile
Technology	C / C++
Version	1.0
Team Members	Rohan Komperla - PES1UG24CS380 / Rujul Jain - PES1UG24CS388 / Shana Susan Varghese - PES1UG24CS427 / M Ajjaiah - PES1UG25CS823

1. Introduction

1.1 Purpose

This document specifies the functional, non-functional and security requirements for the Traffic Management System (Simulation). The system is a C/C++ based simulation that models vehicle movement through roads and intersections, traffic signal strategies, emergency vehicle priority and traffic incidents.

1.2 Intended Audience

This SRS is intended for the development team, the course faculty/evaluators, and anyone who maintains the project after this phase.

1.3 Product Scope

The system provides a controlled simulation environment to model traffic flow and evaluate fixed-time and adaptive signal strategies. It measures traffic performance indicators such as queue length, waiting time and throughput, and allows different strategies to be compared.

1.4 References

- Team 11 Synopsis / Project Proposal - Traffic Management System (Simulation)
- PES University Software Engineering Level 3 - Project Guidelines 2026

2. Overall Description

2.1 Product Perspective

The system is a standalone simulation application. It does not control real traffic-signal hardware and does not integrate with live traffic sensors or emergency services. All road, vehicle and signal data used by the system is generated or configured within the simulation.

2.2 Product Functions

- Configure road network, lanes and intersections
- Generate vehicles under configurable traffic density
- Simulate vehicle movement and queue formation
- Operate fixed-time and adaptive traffic signal strategies
- Give emergency vehicles priority at signals
- Simulate traffic incidents such as road blockages
- Monitor live traffic conditions during a run
- Generate performance analytics and compare strategies

2.3 User Classes and Characteristics

User Class	Characteristics
Simulation Operator	Configures scenarios, starts/stops runs, introduces incidents. Frequent user during testing.
Traffic Administrator	Configures road network, intersections, signals and strategies. Higher-privilege user.
Traffic Analyst	Reviews traffic metrics and compares strategy performance. Read-focused user.
Development Team	Builds, tests, validates and maintains the simulation.

2.4 Operating Environment

The system runs as a desktop application on a standard Linux/Windows machine, built with C++17 and CMake, with a terminal or lightweight graphical output for simulation state and statistics.

2.5 Design and Implementation Constraints

- Implementation language restricted to C/C++
- CMake used as the build system
- Agile methodology followed throughout development
- Git/GitHub used for version control; Jira used for backlog and sprints

2.6 Assumptions and Dependencies

- All traffic data (vehicle arrival, density, incidents) is simulated, not real-world data
- GoogleTest is available for unit testing
- The simulation runs on a single machine in a single process

3. External Interface Requirements

3.1 User Interfaces

A terminal-based or simple graphical interface displays the road network, live vehicle/queue state, signal states and end-of-run statistics. Configuration is done through a config file or command-line options.

3.2 Software Interfaces

The system does not depend on external databases or third-party services. Internal modules communicate through well-defined C++ class interfaces.

3.3 Communications Interfaces

Not applicable. The simulation runs as a single local process with no network communication.

4. Analysis Models

Use Case Diagram covering all actors and use cases listed in Section 9, and a Class Diagram covering the core domain classes: Road, Intersection, TrafficSignal, Vehicle, IncidentManager and AnalyticsEngine. Diagrams to be added during the Design phase.

5. System Features

Features are grouped by the functional areas identified in the synopsis (F1-F8). Each functional requirement below is uniquely tagged, concise, verifiable and traceable to a use case.

ID	Requirement	Priority	Verification
FR-001	The system shall allow configuration of roads, lanes and intersections before a run starts.	High	Functional test
FR-002	The system shall allow configuration of traffic signals at each intersection.	High	Functional test
FR-003	The system shall generate vehicles according to a configurable traffic-density setting.	High	Functional test
FR-004	The system shall move vehicles along assigned paths and place them in a queue when movement is restricted.	High	System test
FR-005	The system shall support a fixed-time signal strategy with configurable cycle timings.	High	Functional test
FR-006	The system shall support an adaptive signal strategy that adjusts timing based on measured traffic density.	High	Functional test
FR-007	The system shall identify simulated emergency vehicles and grant them signal priority.	High	Integration test
FR-008	The system shall allow a simulated incident to reduce or block capacity on a selected road.	Medium	Functional test
FR-009	The system shall record queue length, waiting time and throughput during a run.	High	Unit + system test
FR-010	The system shall generate summary statistics at the end of a simulation run.	High	Functional test
FR-011	The system shall allow the same scenario to be run under different strategies for comparison.	Medium	System test
FR-012	The system shall reject an invalid or incomplete network configuration with a descriptive error.	Medium	Negative test

6. Non-Functional Requirements

ID	Category	Measurable Requirement	Verification
NFR-001	Performance	The simulation shall process at least 1000 vehicle movement steps within 2 seconds on the reference test machine.	Performance test
NFR-002	Reliability	A simulation run shall complete without abnormal termination for any valid configuration.	System test
NFR-003	Maintainability	Core simulation logic shall be separated into independent modules with automated unit tests.	Code review
NFR-004	Testability	Core modules (signal control, vehicle movement, analytics) shall be testable in isolation using mocks/stubs.	Unit test
NFR-005	Usability	An operator shall be able to start a basic simulation run using no more than 3 configuration steps.	Usability walkthrough

7. Security Requirements

ID	Security Requirement	Validation
SEC-001	The system shall validate all configuration file inputs before use.	Negative/security
SEC-002	The system shall reject configuration values outside defined safe ranges (e.g. negative densities).	Negative test
SEC-003	The system shall avoid unsafe memory operations (buffer overflows, unchecked pointers) in core modules.	Static analysis
SEC-004	The system shall log invalid configuration attempts for later	System test

8. Business Rules / Domain Rules

- An emergency vehicle always receives priority over the normal signal cycle at its intersection.
- A simulated incident can only be placed on a road that exists in the configured network.
- Only the Traffic Administrator role may modify the road network or signal configuration during setup.
- Adaptive mode may only use traffic-density data collected within the current run.
- A vehicle cannot proceed through an intersection when its signal is red, except an emergency vehicle under priority rules.

9. Use Cases

ID	Use Case	Primary Actor	Precondition	Outcome
UC-01	Configure Road Network	Administrator	Network is not yet defined	Roads, lanes and intersections configured
UC-02	Configure Traffic Signals	Administrator	Road network exists	Signal states and strategy configured
UC-03	Start Simulation	Operator	Configuration complete	Simulation begins execution
UC-04	Generate Traffic	System	Simulation running	Vehicles generated per density setting
UC-05	Simulate Vehicle Movement	System	Vehicles exist	Vehicles move / queue per signal state
UC-06	Manage Adaptive Signals	Traffic Management Module	Adaptive mode enabled	Signal timing adjusted to density
UC-07	Handle Emergency	Operator / System	Emergency vehicle active	Vehicle receives signal priority
UC-08	Simulate Traffic Incident	Operator	Simulation running	Selected road capacity reduced/blocked
UC-09	Monitor Traffic	Analyst / Operator	Simulation running	Live traffic state displayed
UC-10	View Simulation Analytics	Analyst	Run completed	Performance metrics and comparison shown

10. Validation Specification - Initial Planning

At this stage, validation planning identifies how each requirement category will eventually be validated. This is planning only; execution and results are reported later in the project.

Requirement Category	Planned Validation	Example
Functional	Black-box functional tests	Configure network, run fixed-time strategy
Security	Input validation / negative tests	Invalid config file, out-of-range density
Performance	Load / timing test	1000+ vehicle-step run under time limit

Requirement Category	Planned Validation	Example
Reliability	Long-run / stress scenarios	Extended run with high density, no crash
Usability	Task-based walkthrough	Operator configures and starts a basic run
Unit	Automated unit tests + coverage	Signal timing calculation, queue operations

11. Project Plan

Sprint	Planned Activities	Acceptance Evidence
Sprint 1	Requirements, GitHub/Jira setup, project structure	SRS baseline, repo, backlog
Sprint 2	Road network and vehicle-flow simulation	Working network + vehicle flow, unit tests
Sprint 3	Traffic signal control (fixed + adaptive)	Working signal strategies, test evidence
Sprint 4	Emergency priority and incident management	Integrated features, validation evidence
Sprint 5	Analytics and module integration	Integrated simulation, performance results
Sprint 6	Validation, CI/CD, static analysis, demo prep	Final validation evidence, release build

11.1 Tool Stack

Activity	Tool	Purpose
Programming	C++17	Implementation of simulation modules
Build System	CMake	Build configuration
Version Control	Git + GitHub	Source control, branches, PRs
Agile Planning	Jira	Backlog, user stories, sprints
Unit Testing	GoogleTest	Automated unit testing
CI/CD	Jenkins	Automated build and test pipeline
Containerization	Docker	Reproducible build environment
Static Analysis	SonarQube or similar	Code quality checks

Activity	Tool	Purpose
Design	Draw.io / PlantUML	UML diagrams

Appendix A: Glossary

Term	Definition
Queue	Ordered list of vehicles waiting at an intersection for signal clearance
Fixed-time strategy	Signal strategy where green/red durations are pre-set and do not change
Adaptive strategy	Signal strategy where timing changes based on measured traffic density
Throughput	Number of vehicles that pass through an intersection in a given time
Incident	A simulated event that reduces or blocks road capacity

Appendix B: Field Layouts

Field	Length	Data Type	Description	Mandatory
road_id	10	String	Unique road identifier	Y
intersection_id	10	String	Unique intersection identifier	Y
signal_state	10	Enum	RED / GREEN / YELLOW	Y
vehicle_id	10	String	Unique vehicle identifier	Y
vehicle_type	15	Enum	NORMAL / EMERGENCY	Y
queue_length	5	Integer	Number of vehicles waiting	Y
wait_time	8	Float	Time spent waiting, in seconds	Y
incident_id	10	String	Unique incident identifier	N

Appendix C: Requirement Traceability Matrix

To be completed during the Design phase, once architecture and code references are available.

Req ID	Brief Description	Architecture Ref	Design Ref	Code File Ref	Test Case ID	System Test ID
FR-001	Configure road network					
FR-005	Fixed-time signal strategy					
FR-006	Adaptive signal strategy					
FR-007	Emergency vehicle priority					
FR-009	Traffic metrics collection					